The Gradient Descent Approach to Inverse Kinematics

Robotic Arm Course · Lesson 7

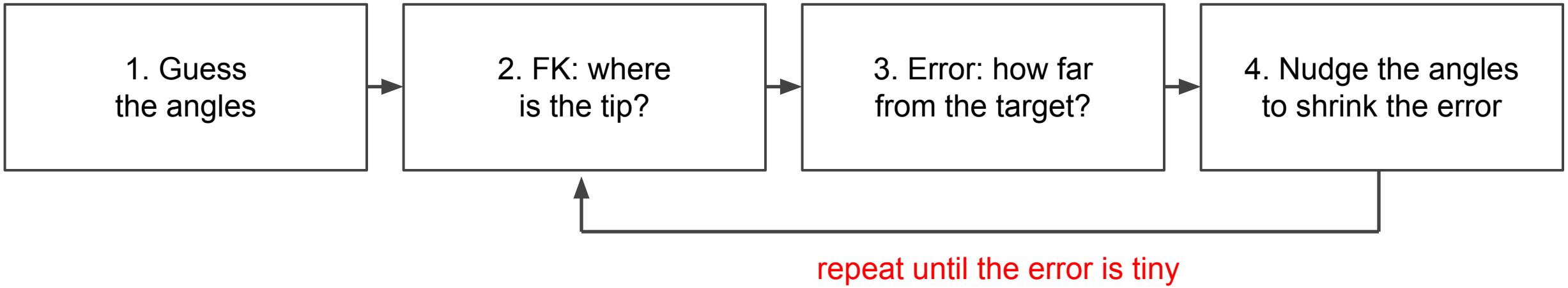
Day 1: Gradient descent by hand
Day 2: The 3D arm

Where Lesson 6 left off:

Target (301.78, 301.78, 176.78)
Triangles gave $\theta_B = 45°$, $\theta_S = 45°$, $\theta_E = -45°$ exactly.

Today's method needs only one thing we already have: forward kinematics. Instead of solving for the angles, we improve a guess until the tip lands on the target.

Guess, check, improve:



The clever part is step 4: which way should each joint turn, and how far? Gradient descent answers both.

The robot never needs a formula for the angles. It only needs to compute where the tip is (FK), again and again.

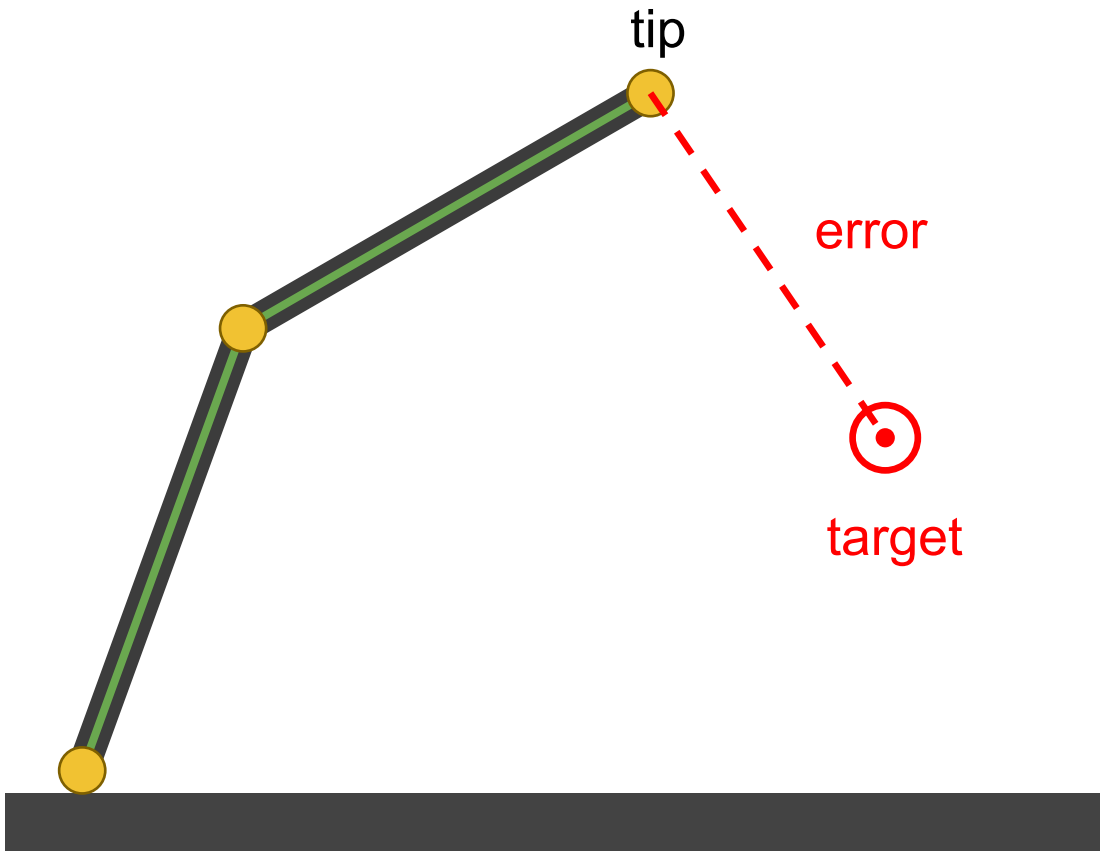
Measuring the error:

The **error** is the distance from the tip to the target, in mm.

We score a guess with the **cost**:
C = (error ÷ 250)²
Dividing by 250 mm measures the error in link lengths, so the numbers stay small.

Why squared? It is never negative, it is zero only on the target, and it makes a smooth bowl to roll down.

Goal: find angles that make C as small as possible.



Rolling downhill:

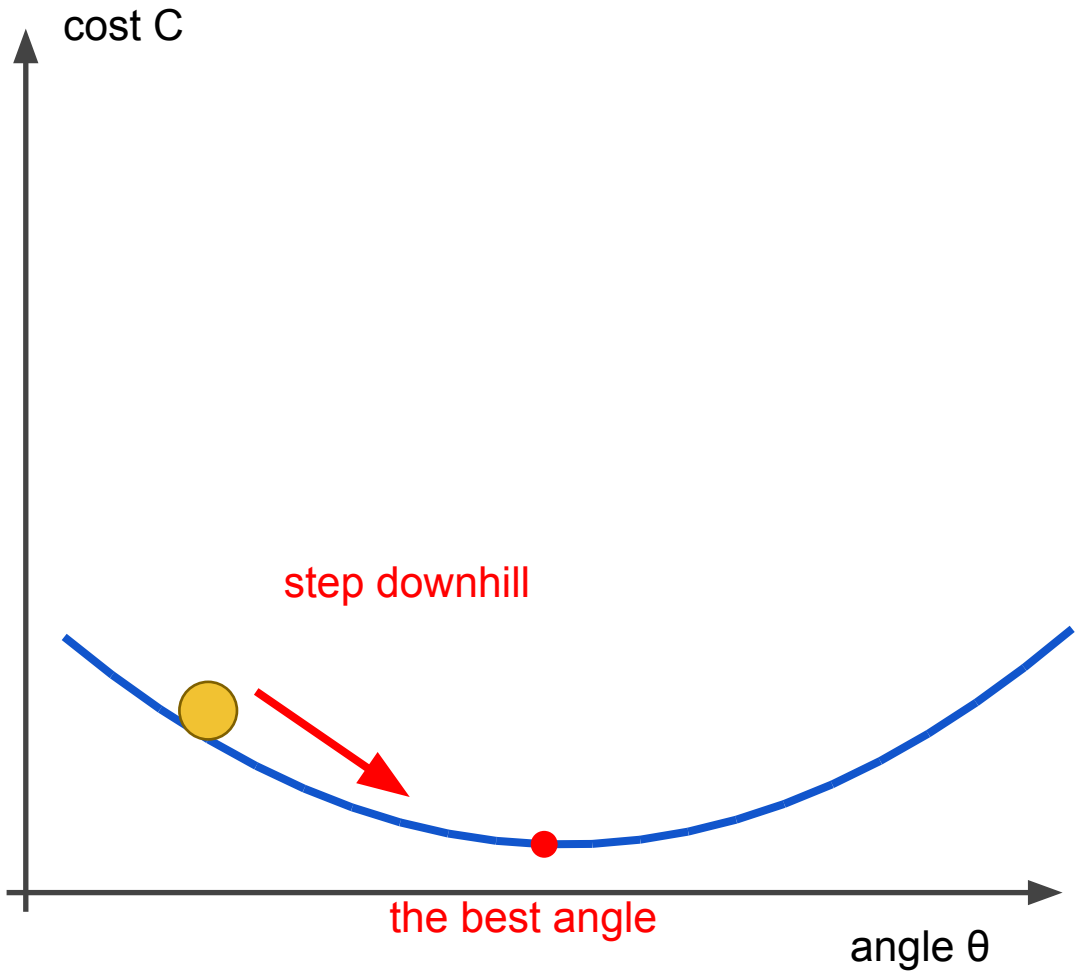
Plot the cost against an angle. It looks like a valley. The best angle is at the bottom.

1. Measure the **slope** where you stand.
2. Step the opposite way (downhill):

$\theta_{new} = \theta - \eta \times \text{slope}$

η (eta) is the **learning rate**: the size of each step.

Slope negative (going down to the right): θ gets bigger. Slope positive: θ gets smaller.



Worked example: one number

Find the lowest point of $f(x) = (x - 3)^2$. (It is at $x = 3$.)
We give you the slope: $2(x - 3)$. Start at $x = 0$ with $\eta = 0.1$.

Step	**x**	**slope $2(x - 3)$**	**new $x = x - 0.1 \times$ slope**
1	0	−6	0.6
2	0.6	−4.8	1.08
3	1.08	−3.84	1.464
4	1.464	−3.072	1.7712
5	1.7712	−2.4576	2.017

The steps shrink as the slope gets flatter near the bottom: 0.6, then 0.48, then 0.38 …
We never jump straight to 3, but we get closer every step.

Try it yourself #1:

Same function: $f(x) = (x - 3)^2$, slope $2(x - 3)$, start at $x = 0$.
Take two steps with each learning rate:

(a) $\eta = 0.25$ (b) $\eta = 1$

Try it yourself #1: solution

Same function: $f(x) = (x - 3)^2$, slope $2(x - 3)$, start at $x = 0$.
Take two steps with each learning rate:

(a) $\eta = 0.25$ (b) $\eta = 1$

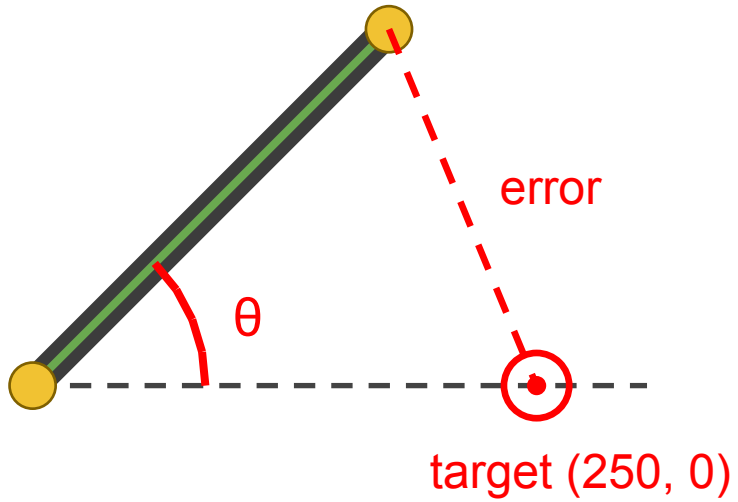
(a) $x = 0 - 0.25 \times (-6) = 1.5$
$x = 1.5 - 0.25 \times (-3) = 2.25$
Bigger steps: closer to 3 than $\eta = 0.1$ after two steps.

(b) $x = 0 - 1 \times (-6) = 6$
$x = 6 - 1 \times 6 = 0$, then 6, 0, 6 … forever.
The step is too big. It jumps over the bottom every time and never settles.

Gradient descent on one joint:

One 250 mm link at angle θ from the floor. The target is (250, 0), so the answer is θ = 0°.
The tip is (250 cos θ, 250 sin θ), which gives
$C = 2 - 2\cos\theta$ and slope $= 2\sin\theta$.
Start at θ = 45° with η = 0.2.

Step	θ	slope	new θ (rad)	new θ
1	45.00°	1.4142	0.5026	28.79°
2	28.79°	0.9633	0.3099	17.76°
3	17.76°	0.6099	0.1879	10.77°
4	10.77°	0.3736	0.1132	6.49°
5	6.49°	0.2259	0.0680	3.90°



The step rule uses radians: 180° = π ≈ 3.1416 rad. Change back to degrees at the end (× 180 ÷ π).

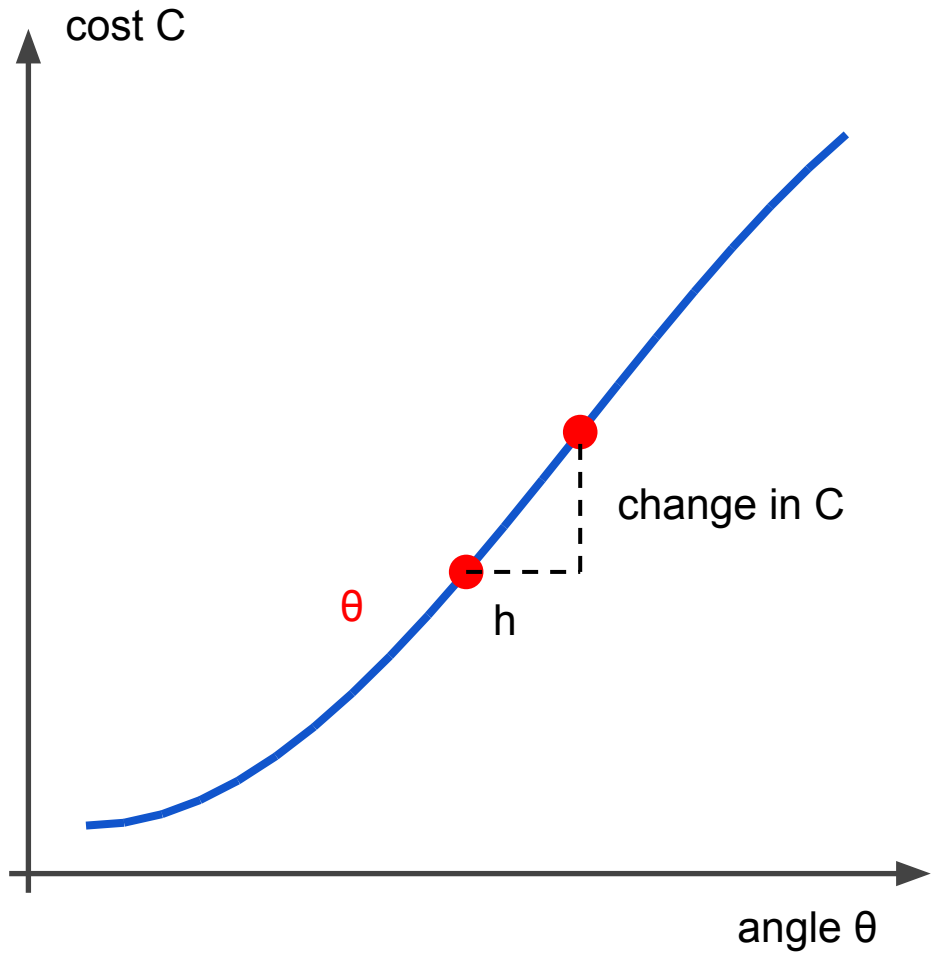
Measuring the slope by nudging:

We do not need calculus. **Nudge** the joint a tiny amount h and watch the cost change:
slope ≈ (C(θ + h) − C(θ)) ÷ h

At θ = 45° (0.7854 rad), with h = 0.01:
C(θ) = 0.58579
C(θ + h) = 0.60000
slope ≈ (0.60000 − 0.58579) ÷ 0.01 = 1.42

The exact slope is 2 sin 45° = 1.414. Very close!

This only needs the cost, and the cost only needs FK. So it works for any arm.



Try it yourself #2:

One joint again: $C = 2 - 2\cos\theta$ and slope $= 2\sin\theta$, with $\eta = 0.2$.
Start at $\theta = 30°$ and take two steps. Give θ in degrees.

Try it yourself #2: solution

One joint again: $C = 2 - 2\cos\theta$ and slope $= 2\sin\theta$, with $\eta = 0.2$.
Start at $\theta = 30°$ and take two steps. Give θ in degrees.

Step 1: slope $= 2\sin 30° = 1$

$\theta = 0.5236 - 0.2 \times 1 = 0.3236$ rad $= 18.54°$

Step 2: slope $= 2\sin 18.54° = 0.6360$

$\theta = 0.3236 - 0.2 \times 0.6360 = 0.1964$ rad $= 11.25°$

Each step moves closer to 0°, and each step is smaller than the last.

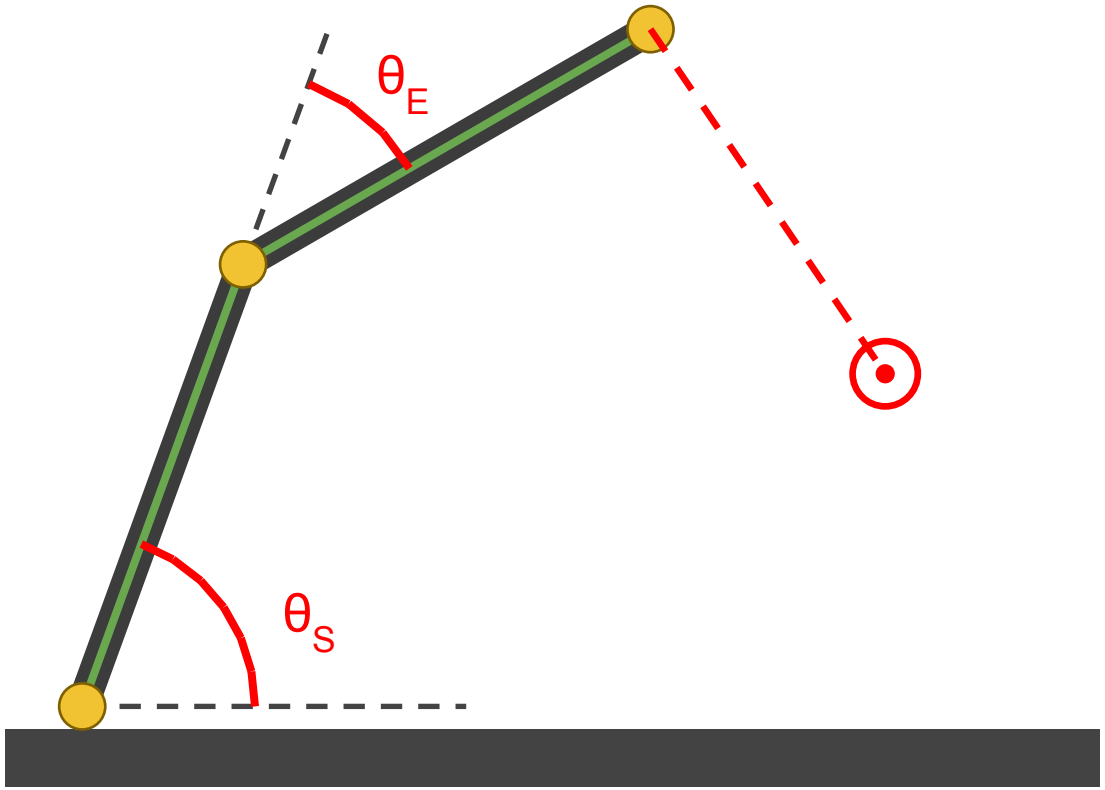
The flat arm: two joints

Now the cost depends on two angles, θ_S and θ_E. FK from Lesson 5 gives the tip: flat_tip(θ_S, θ_E).

The **gradient** is two slopes, one per joint:
nudge θ_S (keep θ_E) and see how C changes,
then nudge θ_E (keep θ_S) and see how C changes.

Then both angles take a step downhill at the same time:

$\theta_S = \theta_S - \eta \times \text{slope}_S$
$\theta_E = \theta_E - \eta \times \text{slope}_E$



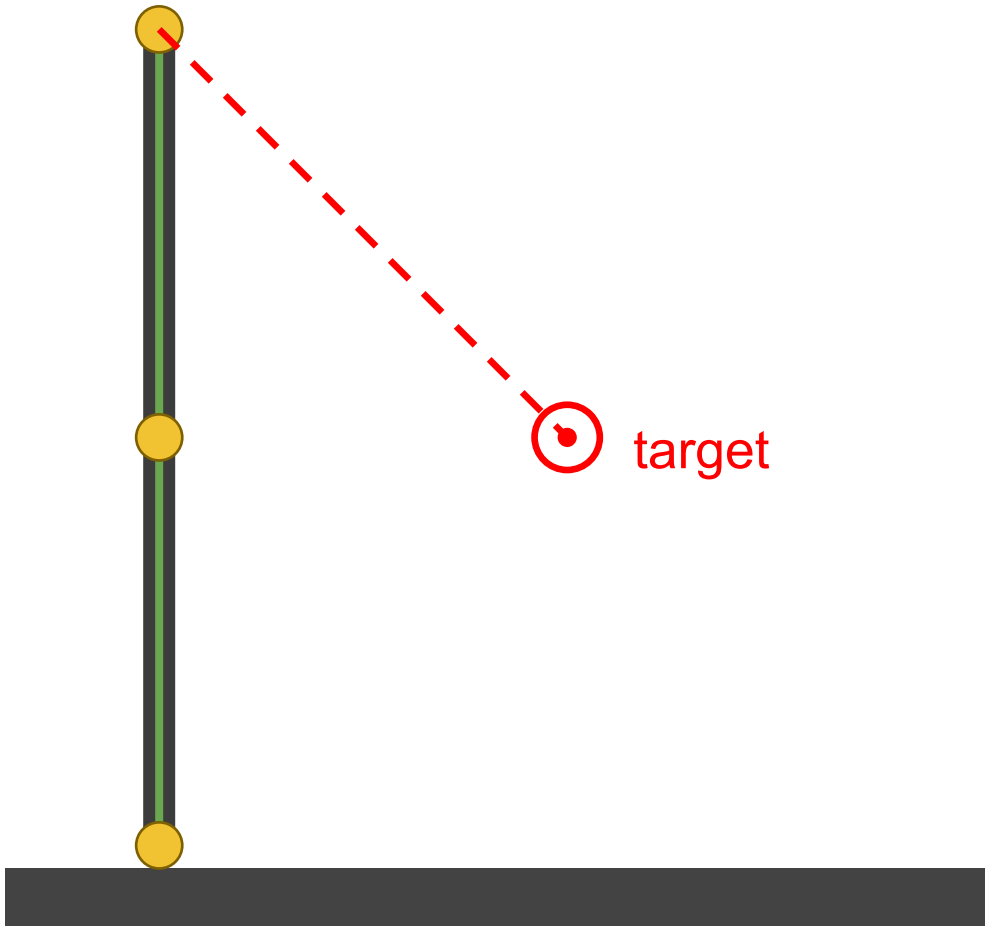
Worked example: set up

Target: (250, 250), with $L_1 = L_2 = 250$ mm.

Start: $\theta_S = 90°$, $\theta_E = 0°$. The arm points straight up and the tip is at (0, 500).

Learning rate: $\eta = 0.1$

The tip is 250 mm left of the target and 250 mm above it:
error = $\sqrt{(250^2 + 250^2)} = 353.6$ mm
$C = (353.6 \div 250)^2 = 2$



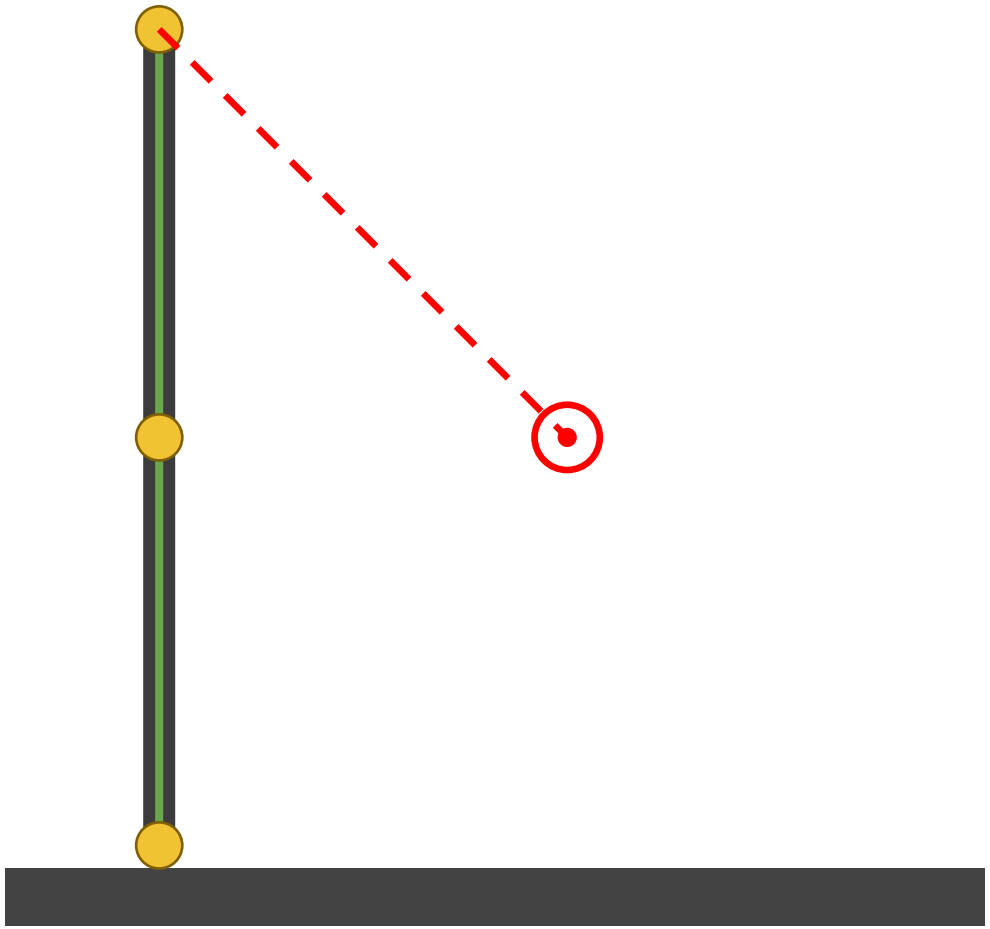
Worked example: measure the slopes

Nudge each joint by h = 0.01 and see what the cost does. At the start, C = 2.

Nudge θ_S: C = 2.0402
$\text{slope}_S \approx (2.0402 - 2) \div 0.01 \approx 4$

Nudge θ_E: C = 2.0200
$\text{slope}_E \approx (2.0200 - 2) \div 0.01 = 2$

Both slopes are positive: raising either angle makes the cost bigger. So both angles must go down. θ_S matters twice as much as θ_E.



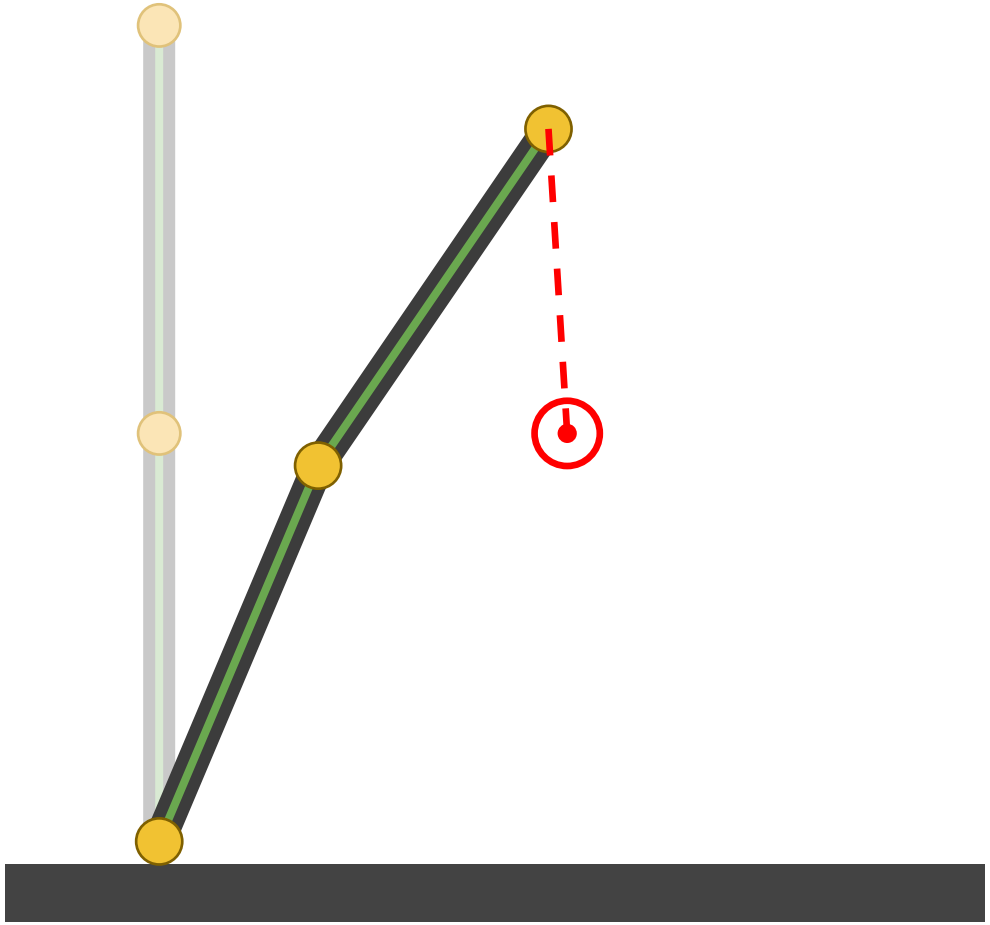
Worked example: take a step

Update both joints at once, with $\eta = 0.1$:

$\theta_S = 1.5708 - 0.1 \times 4 = 1.1708$ rad = 67.08°
$\theta_E = 0 - 0.1 \times 2 = -0.2$ rad = −11.46°

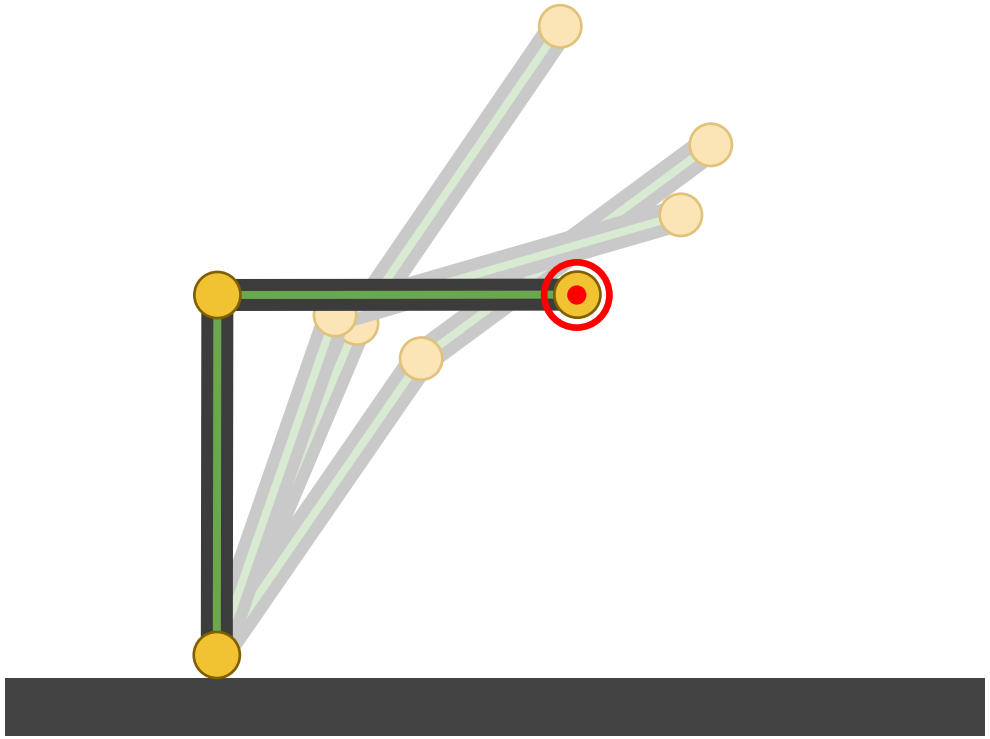
The tip moves from (0, 500) to (238.5, 436.6).
The error drops from 353.6 mm to 187.0 mm.

Then repeat: measure new slopes, take another step.



Worked example: keep going

Step	θ_S	θ_E	tip	error (mm)
0	90.00°	0.00°	(0.0, 500.0)	353.6
1	67.08°	−11.46°	(238.5, 436.6)	187.0
2	58.00°	−16.72°	(320.3, 376.9)	145.1
3	55.41°	−18.97°	(343.1, 354.3)	139.8
10	57.29°	−26.51°	(349.9, 338.3)	133.3
30	70.82°	−54.67°	(322.3, 305.6)	91.2
60	86.72°	−84.54°	(264.1, 259.1)	16.8
106	89.91°	−89.85°	(250.4, 250.3)	0.5



Big steps at first, then smaller ones. After 106 steps the error is under 0.5 mm, at $\theta_S = 89.91°$, $\theta_E = -89.85°$.

Lesson 6 gave $\theta_S = 90°$, $\theta_E = -90°$ for this target.

Same answer, no triangles:

Lesson 6: trigonometry
$\theta_S = 90°, \theta_E = -90°$
The law of cosines, atan2 and a few formulas.
Exact, in one pass. Needs the triangles.

Lesson 7: gradient descent
$\theta_S = 89.91°, \theta_E = -89.85°$
Guess, nudge, step, repeat (106 steps).
Very close. Needs only FK and a loop.

Same target, almost the same pose.

Gradient descent trades exactness for generality: if you can compute FK for an arm, you can do IK for it.

Try it yourself #3:

Target (341.51, 341.51). Start at $\theta_S = 90°$, $\theta_E = 0°$ with $\eta = 0.1$.
The slopes at the start are $slope_S = 5.46$ and $slope_E = 2.73$.
Take one step. Give θ_S and θ_E in degrees.

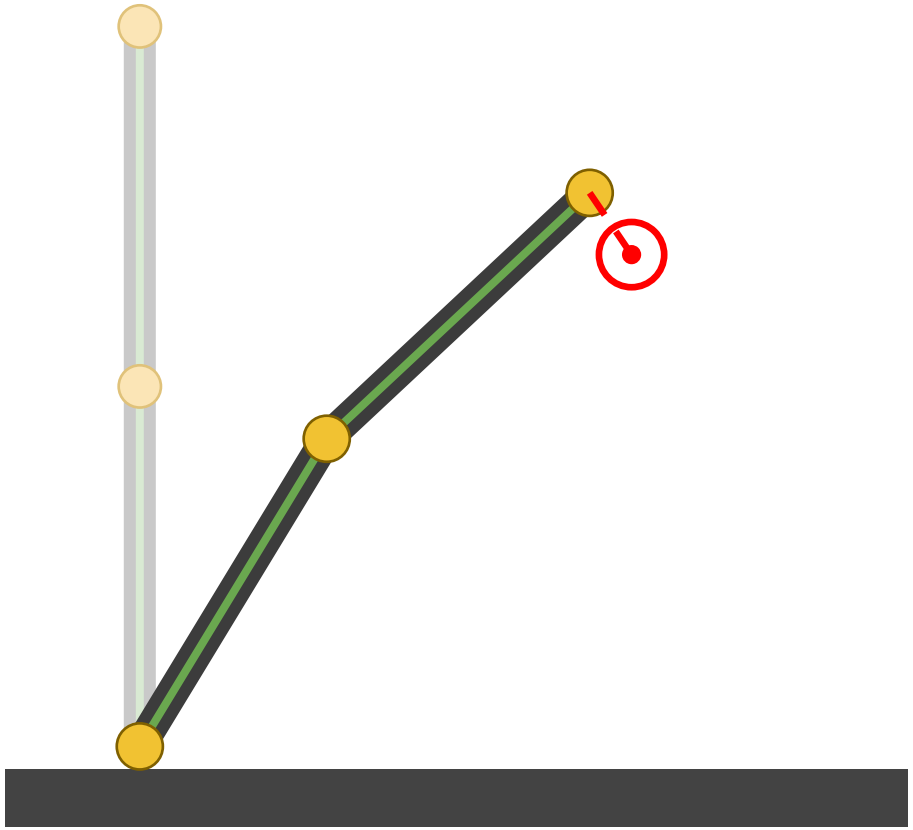
Try it yourself #3: solution

Target (341.51, 341.51). Start at $\theta_S = 90°$, $\theta_E = 0°$ with $\eta = 0.1$.
The slopes at the start are $\text{slope}_S = 5.46$ and $\text{slope}_E = 2.73$.
Take one step. Give θ_S and θ_E in degrees.

$\theta_S = 1.5708 - 0.1 \times 5.46 = 1.0248$ rad = 58.72°
$\theta_E = 0 - 0.1 \times 2.73 = -0.273$ rad = −15.64°

The tip moves to (312.4, 384.4). The error falls from 376.5 mm to 51.8 mm in one step.

Lesson 6's answer is $\theta_S = 60°$, $\theta_E = -30°$, so we are already close.



Day 1 recap:

1. Cost: $C = (\text{error} \div 250)^2$. A bowl whose bottom is the answer.
2. Slope: nudge a joint and watch the cost: $(C(\theta + h) - C(\theta)) \div h$. Only FK is needed.
3. Step: $\theta_{new} = \theta - \eta \times \text{slope}$, for every joint at once. The update uses radians.
4. Repeat until the error is tiny.

Tomorrow the base turns too, so there are three joints to nudge.

Day 2: where Day 1 left off

1. What does the learning rate η control?

2. The slope is positive. Does θ go up or down?

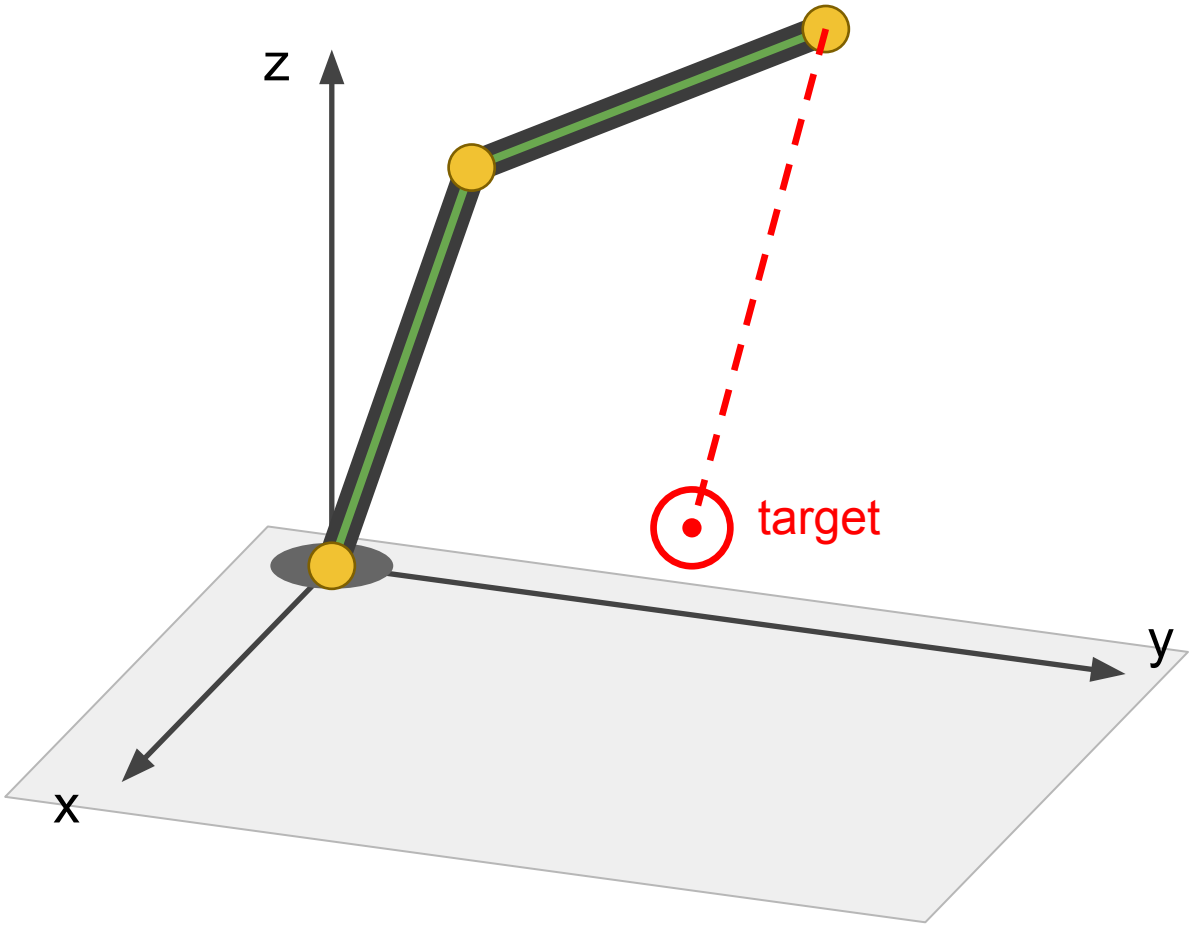
3. Why do we nudge a joint instead of using a formula for the slope?

Extending to 3D:

Now the arm has three joints: θ_B, θ_S and θ_E. Nothing in the method changes:

1. FK: tip_3d from Lesson 5 gives the tip.
2. Cost: C = (error ÷ 250)², with error in 3D.
3. Gradient: three slopes, by nudging θ_B, θ_S and θ_E in turn.
4. Step all three downhill, then repeat.

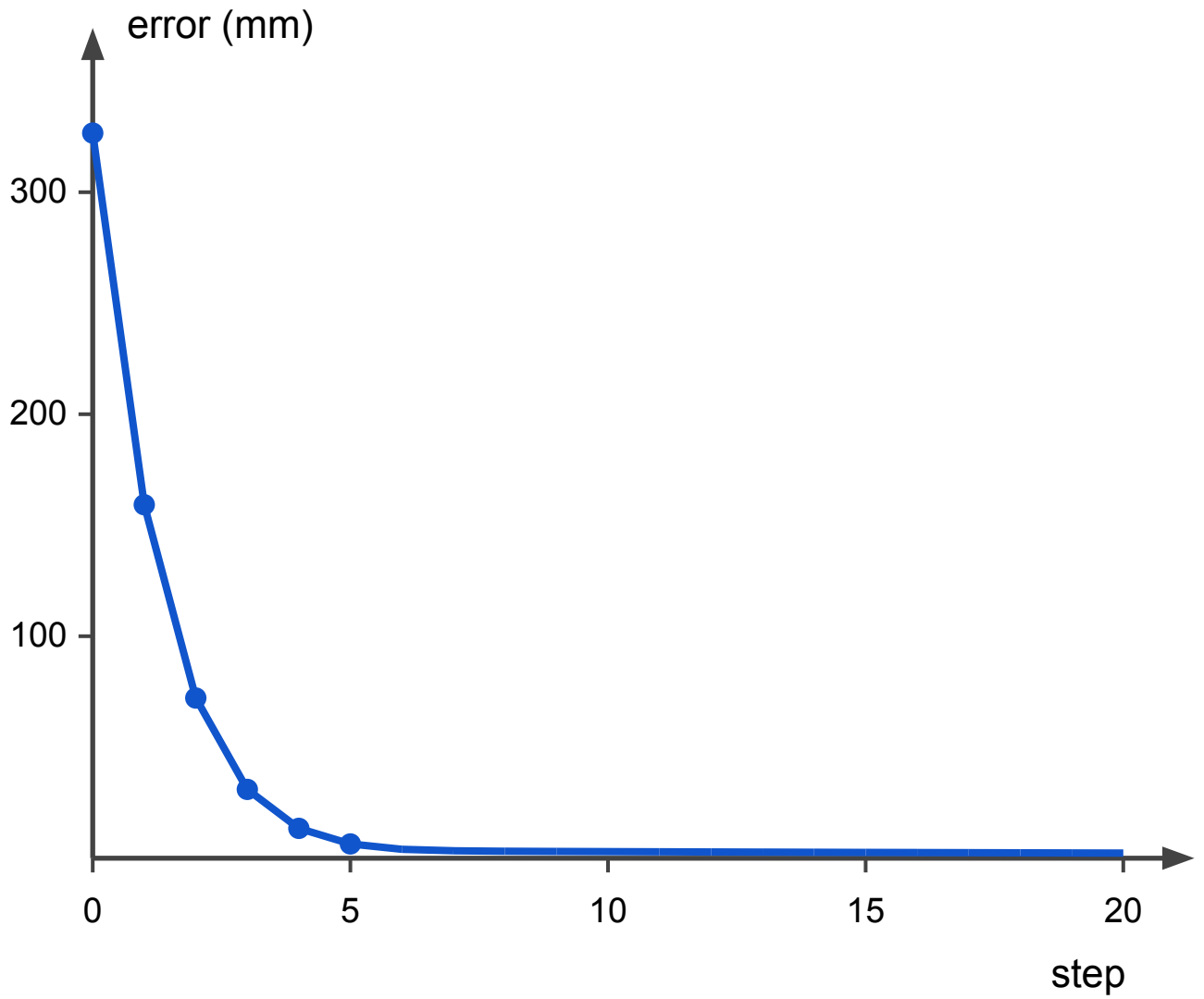
By hand this is a lot of arithmetic, so a computer ran the loop. For the target (301.78, 301.78, 176.78), it stopped after 84 steps at (45.0°, 45.15°, −45.29°), 0.49 mm from the target and within 0.3° of Lesson 6.



Watching the error fall:

Step	error (mm)
0	326.64
1	159.23
2	72.17
3	31.00
5	6.43
10	2.96
40	1.41
84	0.49

Fast at first, then a slow crawl.



The first 20 steps. From step 20 it takes 64 more steps to go from 2.31 mm to 0.49 mm: the valley is flat near the bottom, so the slopes and the steps get tiny.

Choosing the learning rate η:

Same target (301.78, 301.78, 176.78), same start, five learning rates:

η	**steps to reach 0.5 mm**
0.02	364
0.05	153
0.1	84
0.2	52
0.3	never settles (bounces between 234.5 and 288.0 mm)

Too small: safe but slow. Too big: it overshoots and bounces, like η = 1 in Try it yourself #1.
We use η = 0.1: fast enough, and safe for every target in this lesson.

When do we stop?

The loop needs a rule, or it could run forever.

1. **Tolerance**: stop when the error is small enough. We use 0.5 mm.

2. **Step limit**: stop after a set number of steps (1000), even if the error is still big.

Stopping at the step limit does not mean the answer is right. Always look at the final error.

A real robot also has a time budget, so it needs the step limit.

Try it yourself #4:

A computer ran gradient descent for the target (295.75, 170.75, 91.51), from the start (0°, 45°, −45°) with $\eta = 0.1$. It stopped after 79 steps at (30.0°, 59.91°, −89.85°), with an error of 0.49 mm.
Did it reach the target? Compare with Lesson 6's Try it yourself #4.

Try it yourself #4: solution

A computer ran gradient descent for the target (295.75, 170.75, 91.51), from the start (0°, 45°, −45°) with η = 0.1. It stopped after 79 steps at (30.0°, 59.91°, −89.85°), with an error of 0.49 mm.
Did it reach the target? Compare with Lesson 6's Try it yourself #4.

Yes. The error, 0.49 mm, is under the 0.5 mm tolerance, so the loop stopped long before the 1000-step limit.

Lesson 6 gave (30°, 60°, −90°): the same pose, to within 0.2°.

With η = 0.2 the same run needs only 39 steps (error 0.43 mm).

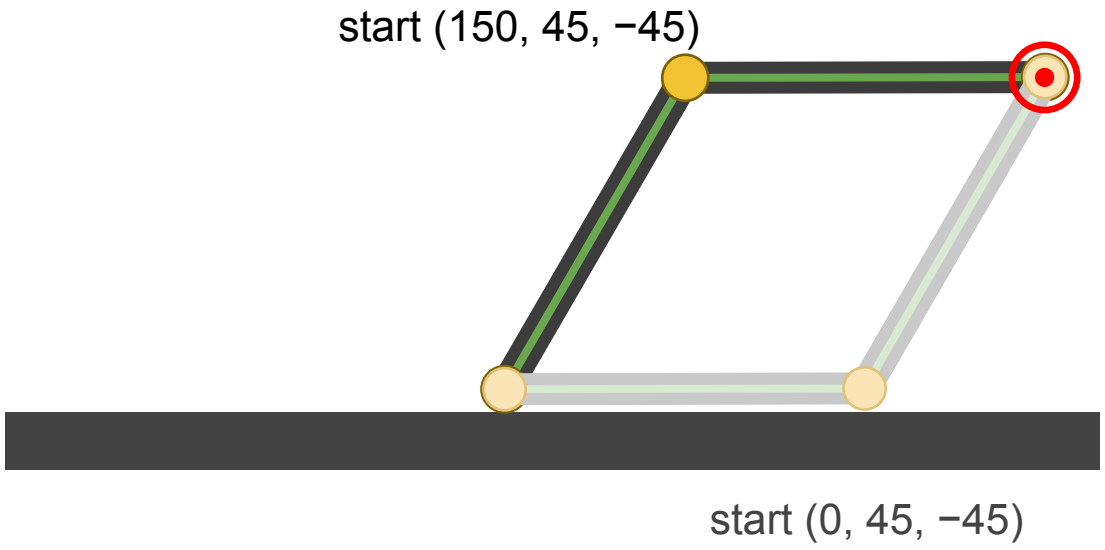
Different start, different answer:

Target (−324.76, 187.50, 216.51): Lesson 6's #5, answer (150°, 60°, −60°).

Start (0, 45, −45):
[−30.0, 179.88, −59.78]
The base points away, and the arm reaches back over the top.

Start (150, 45, −45):
[150.0, 59.88, −59.77]

Both put the tip on the target. Gradient descent rolls into the nearest valley, so start near the pose you want, and check the joint limits.



side view, looking along the base direction 150°

Try it yourself #5:

A computer ran gradient descent twice for the target (−125, 216.51, 250). Lesson 6 gives (120°, 90°, −90°).
Start (0°, 45°, −45°): (120.0°, 89.9°, −89.84°) in 73 steps.
Start (0°, 90°, 0°): (−60.0°, 90.1°, 89.84°) in 106 steps.
Are both answers right? Why did the base turn a different way?

Try it yourself #5: solution

A computer ran gradient descent twice for the target (−125, 216.51, 250). Lesson 6 gives (120°, 90°, −90°).
Start (0°, 45°, −45°): (120.0°, 89.9°, −89.84°) in 73 steps.
Start (0°, 90°, 0°): (−60.0°, 90.1°, 89.84°) in 106 steps.
Are both answers right? Why did the base turn a different way?

Both are right. Both errors are within the 0.5 mm tolerance, so both tips are on the target.

The second run started straight up, so the arm could fall either way. It found the other valley: the base turned to −60° instead of 120°, and the forearm reaches back.

Choose the answer that fits the robot’s joint limits.

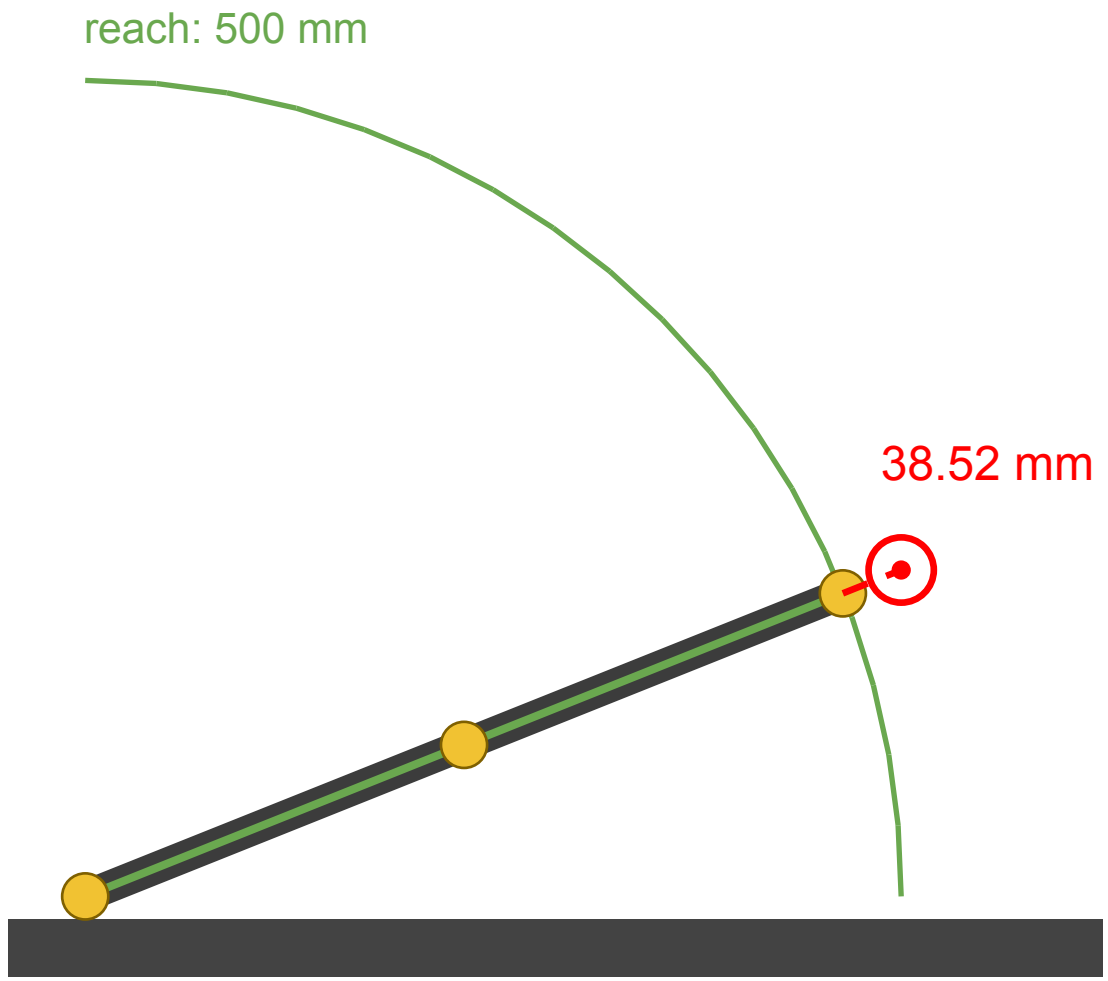
Out of reach: no error, just a gap

Try Lesson 6's unreachable target (400, 300, 200).

Trigonometry found no answer: the cosine came out as 1.32. Gradient descent still gives angles: [36.87, 21.8, 0.0], 1000 steps, error 38.52 mm

It used every step, and stopped with the arm straight ($\theta_E = 0°$), pointing at the target, as close as it can get: 538.52 − 500 = 38.52 mm short.

Always check the final error. Otherwise the robot thinks it got there.



side view, in the arm's plane

When gradient descent struggles:

Situation	**What happens**
Target out of reach	ends with a leftover error (38.52 mm for (400, 300, 200))
Target near full reach	very slow: (375, 216.51, 250) is still 1.5 mm off after 1000 steps
Learning rate too big	bounces and never settles
Learning rate too small	hundreds of steps
Different start	a different (valid) answer

Near full reach the arm is almost straight. Bending the elbow then barely moves the tip toward the target, so the slope is tiny and so are the steps.

Trigonometry vs. gradient descent

Lesson 6: trigonometry

+ Exact answer in one pass
+ Very fast
− New triangles for every new joint
− Special cases by hand

Lesson 7: gradient descent

+ The same loop for any arm: only FK changes
+ Unreachable targets fail gently
− Close, not exact, and slower
− Needs a good η and a good start

If the arm had six joints, which method would you use, and why?

With gradient descent the loop stays the same: only FK changes.

Making it faster:

Plain gradient descent is the simplest numerical IK method. Real robots use the same loop with smarter steps:

1. **Warm start**: begin from the arm's current pose. The next target is usually close, so only a few steps are needed.

2. **Exact slopes**: calculus can give the slopes directly, without nudging (Jacobian methods, Lesson 8).

3. **Adaptive steps**: take big steps where the valley is steep and small ones where it is flat.

All of them are still guess, measure, improve.

What we learned:

1. Iterative IK guesses, measures the error and improves. It only needs FK.
2. Cost = (error ÷ 250)². The slope of each joint comes from nudging it.
3. $\theta_{new} = \theta - \eta \times \text{slope}$. Too big an η bounces, too small an η crawls.
4. Always check the final error, and mind the start pose.

Next time (Lesson 8): we write IK code ourselves and meet the Jacobian.